

Process Discovery & Event Prediction using Deep Learning

A CNN-based Approach to Log Analysis

Siddhant Dani

Memorial University of Newfoundland, St John's NL A1C5S7, Canada

sdani@mun.ca

Abstract. This project explores the application of Convolutional Neural Networks (CNNs) in process mining to enhance the classification of business process traces. Using event logs from the Process Discovery Contest 2023, I set out to create CNN models capable of predicting the next event in a sequence of business activities, primarily as a tool to facilitate trace classification. The process involved meticulous data cleaning, model training, and a rigorous evaluation phase. The results were promising, with an average training accuracy of 70.65% across 96 models. The real test was evaluating these models on new, unseen data, where they demonstrated strong performance, achieving a precision of 0.6875, a recall of 0.6652, and an F1-score of 0.6680. These results underscore the models' reliability and adaptability to various business scenarios, proving their effectiveness in accurately classifying traces.

Keywords: Process Mining, Trace Classification, Deep Learning, Next-Event Prediction

1 Introduction

The complexity of modern business processes, coupled with the exponential growth of digital trace data, has positioned process mining as a crucial discipline for organizations striving to optimize their operations. By bridging the gap between traditional data mining and business process management, process mining offers tools that uncover, monitor, and enhance real-world processes by analyzing event log data. The ultimate goal is to extract actionable insights that drive better decision-making, improve performance, and ensure compliance.

In recent years, integrating deep learning techniques into process mining has garnered significant attention, especially in event log analysis and predictive process monitoring. Techniques such as Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks, renowned for their effectiveness in handling sequence data, have been employed to predict upcoming events, classify event sequences, and uncover underlying process models. This synergy between deep learning and process mining opens up promising avenues for enhancing the accuracy and efficiency of process analysis tasks.

However, applying deep learning to process mining has its challenges. Key issues include

- managing the variability inherent in event logs,
- addressing incomplete or noisy data and
- ensuring that models generalize effectively across diverse processes and datasets.

Moreover, interpreting deep learning models within the context of process mining remains a complex endeavour, often necessitating a hybrid approach that combines traditional process mining techniques with advanced machine learning models.

This project seeks to overcome some of the challenges in applying deep learning to process mining by developing an innovative approach that leverages CNNs and advanced classification techniques for process discovery and event prediction. The proposed methodology is not just innovative, but it is also rigorously evaluated using real-world event logs from the Process Discovery Contest 2023 and 2024, providing empirical evidence of its effectiveness. Additionally, the project explores various strategies for feature engineering, such as utilizing aggregated probabilities and custom kernels for classification, to further enhance model performance.

2 The ICPM Process Discovery Challenge

The ICPM Process Discovery Challenge is an annual contest organized as part of the International Conference on Process Mining (ICPM). The primary aim of the challenge is to advance the field of process mining by providing a platform for researchers, students, and practitioners to apply their skills to real-world data. The challenge focuses on extracting useful insights from event logs, which are data records capturing sequences of activities in various business processes.

Participants are tasked with discovering accurate and understandable process models from the event logs. Furthermore, participants are challenged to classify traces—sequences of activities within the logs—based on patterns, variations, and other characteristics.

The challenge promotes innovation in applying process mining techniques, fostering collaboration among global experts, and encouraging the development of new methodologies and tools. By participating, individuals and teams contribute to the broader understanding and application of process mining in real-world contexts.

3 Prior Work

3.1 Sequence-to-Sequence Models

Sequence-to-sequence models, such as Long Short-Term Memory (LSTM) networks, have proven effective in predicting each step within a sequence. For example, Tax et al. [3] demonstrated that LSTMs excel in predicting various business process aspects, including the next activity, case continuations, and remaining cycle time. However, LSTMs face challenges like handling repeated activity traces, highlighting the need for further optimization. Other studies, like those by Xia et al. [8] and Lu et al. [9], leverage LSTMs for tasks such as predicting process events, correcting missing activity labels, and enhancing process monitoring by integrating spatiotemporal features.

Moreover, studies by Ashrafi et al. [4] and Pishgar et al. [5] extend the application of LSTMs beyond business processes to healthcare, predicting outcomes like mortality rates in patients using electronic health records (EHRs). These models handle complex sequences of patient data, improving accuracy through innovative algorithms and data preprocessing techniques.

3.2 Sequence-to-Vector Models

Sequence-to-vector models, particularly Convolutional Neural Networks (CNNs), are more focused on classifying the sequence or predicting specific outputs. Nicola Di Mauro et al. [1] explored the use of Inception CNNs for predicting the next activity in business processes. Their findings suggest that CNNs, particularly Inception models, can capture both local and global sequence patterns, outperforming LSTMs in terms of computational efficiency.

Other research, like Weytjens and De Weerdts [2], compares CNNs with LSTMs for process outcome prediction, highlighting the superior speed and robustness of CNNs across various hyperparameters. Al-Jebrni et al. [11] further suggest that 1D CNNs offer computational efficiency and better performance compared to traditional Recurrent Neural Networks (RNNs) and LSTMs.

Additionally, hybrid models, such as those proposed by Park and Song [12], combine CNNs with LSTMs for predictive process monitoring. These models convert event logs into image data, which is then processed by CNNs and LSTMs to improve accuracy and efficiency in prediction.

3.3 Other Approaches and Hybrid Models

Several studies have explored hybrid and alternative deep learning methods. Mehdiyev et al. [7] and Evermann et al. [13] introduced multi-stage deep learning models, combining feature hashing, stacked autoencoders, and deep feedforward networks to enhance predictive accuracy. Kamala and Latha [14] explored the use of deep fuzzy neural networks for industrial applications, highlighting the versatility and effectiveness of different deep learning approaches in process mining.

The integration of both sequence-to-sequence and sequence-to-vector models, as well as hybrid approaches, demonstrates the potential of deep learning in process mining. Each method offers unique strengths and addresses specific challenges, indicating that a tailored approach is essential for optimizing performance across various applications.

4 Methodology and Data Processing Framework

4.1 Dataset and Preprocessing

Dataset Overview

The dataset for this project is derived from the Process Discovery Contest 2023 (PDC 2023) and presents unique challenges due to its complexity and variety of event logs. The dataset includes:

- 384 Training Logs: These logs are utilized to train the model and contain sequences of activities representing different instances of a business process.
- 96 Test Logs: These logs are employed to evaluate the performance of the trained model, ensuring it generalizes well to unseen data.
- 96 Base Logs: Serving as a reference, these logs are compared against the test logs to assess the model's predictive capabilities.

Each event log is stored in the IEEE XES format, a standard in process mining for recording and managing event logs.

Data Variations and Noise Levels

The event logs in this dataset are generated from a single base model that allows for several configurable characteristics, leading to different variations in the processes captured. These characteristics include:

- Dependent Tasks: Whether long-term dependencies are present, which affects whether transitions that bypass dependent tasks are enabled.
- Loops: Can be absent, simple, or complex, determining whether transitions that start a loop are present.
- OR Constructs: The presence of OR-joins and OR-splits affects whether certain transitions that take some inputs or generate some outputs are enabled.
- Routing Constructs (Invisible Tasks): Involves the inclusion of invisible tasks that influence the flow of the process without being visible in the logs.
- Optional Tasks: Determines if some tasks can be skipped, adding flexibility to the process.
- Duplicate Tasks (Recurrent Activities): Determines if tasks can be repeated, relabeling some transitions to existing labels.
- Noise: Introduced to simulate real-world scenarios where data may be imperfect.

Noise levels vary:

- No Noise: No alterations in the traces.
 - Moderate Noise: Random events in traces are removed, moved, or copied with certain probabilities.
- Pre-classified Traces: A portion of the traces are pre-labeled as either positive or negative to facilitate model training and evaluation.

These configurable attributes create multiple variations of basic processes and different noise levels, simulating real-world complexities. Each variation aims to test the model's robustness and adaptability under diverse scenarios, reflecting the heterogeneity often encountered in real-world data. The inclusion of noise and pre-classified traces ensures that the models trained are not only accurate but also generalizable across different contexts and data quality levels.

Understanding Event Logs

Event logs play a crucial role in process mining because they capture the detailed execution of business processes. Each log is made up of multiple traces, where each trace represents a sequence of activities (or events) within a single instance of a process. The key components of an event log include:

- Case ID: A unique identifier for a process instance, like an order number. (e.g. trace1)
- Activity: The specific task performed, such as "t1" or "t11."

For example, in an XML-based XES file, a trace might be structured as shown below,

```
<?xml version="1.0" encoding="UTF-8" ?>
<!-- This file has been generated with the OpenXES library. It conforms -->
<!-- to the XML serialization of the XES standard for log storage and -->
<!-- management. -->
<!-- XES standard version: 1.0 -->
<!-- OpenXES library version: 1.0RC7 -->
<!-- OpenXES is available from http://www.openxes.org/ -->
<log xes.version="1.0" xes.features="nested-attributes" openxes.version="1.0RC7">
  <extension name="Concept" prefix="concept" uri="http://www.xes-
standard.org/concept.xesext"/>
    <global scope="event">
      <string key="concept:name" value="__INVALID__"/>
    </global>
    <classifier name="Event Name" keys="concept:name"/>
    <string key="concept:name" value="pdc2023_000000 [1000|1|0]"/>
    <trace>
      <string key="concept:name" value="trace 1"/>
      <event>
        <string key="concept:name" value="t1"/>
      </event>
      <event>
        <string key="concept:name" value="t11"/>
      </event>
      <event>
        <string key="concept:name" value="t12"/>
      </event>
      <event>
        <string key="concept:name" value="t40"/>
      </event>
      <event>
        <string key="concept:name" value="t3"/>
      </event>
      <event>
        <string key="concept:name" value="t5"/>
      </event>
      <event>
        <string key="concept:name" value="t33"/>
      </event>
      <event>
        <string key="concept:name" value="t29"/>
      </event>
    </trace>
  </log>
```

Preprocessing Steps

Preprocessing event logs is an essential step to ensure the data is ready for use in the Convolutional Neural Network (CNN) and other machine learning models. The following steps were taken to clean, standardize, and format the data:

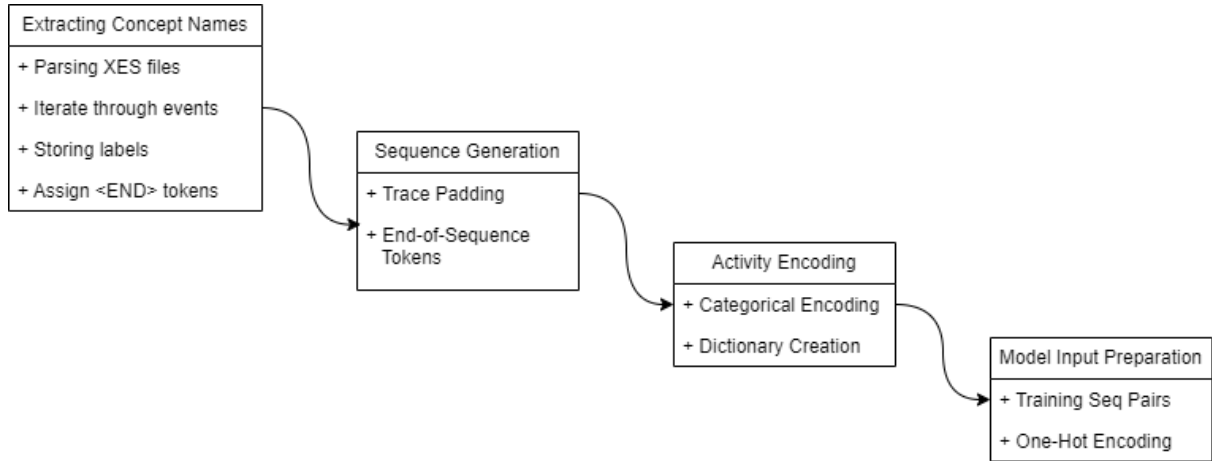


Figure 1: Preprocessing Steps

1. Extracting Concept Names:

- **Parsing XES Files:** The event logs were parsed using Python's `xml.etree.ElementTree` module to navigate through the XML structure.
- **Iterating Through Events:** Each trace and event were processed to extract the `concept:name` attribute, which represents the activity label.
- **Storing Activity Labels:** These labels were collected into sequences that reflect the order of activities in each trace.
- **End-of-Case Marker:** An `<END>` token was added to each sequence to mark the end of the process instance.

These sequences of concept names were then used as input for the model, helping it learn activity patterns and make predictions.

2. Sequence Generation:

- **Trace Padding:** Padding is applied to the beginning of each sequence to achieve a uniform length of 32 events across all traces. This standardization is necessary because deep learning models, such as LSTMs and CNNs, require inputs of consistent size to efficiently process and learn from the data. By ensuring that all sequences have the same length, padding prevents issues that may arise from varying sequence lengths, such as incomplete learning or errors during model training, ultimately contributing to more reliable and accurate predictions.
- **End-of-Sequence Tokens:** An `<END>` token was added to each trace to signal its completion, helping the model distinguish between ongoing and completed sequences.

3. Activity Encoding:

- **Categorical Encoding:** Each unique activity in the logs was converted into a numerical format through categorical encoding. This step involved mapping each activity name to a unique integer, which the CNN could then process as input.

- **Dictionary Mapping:** A dictionary was created to map each activity to its corresponding integer code. This mapping was crucial for both training the model and interpreting predictions during testing.

4. Model Input Preparation:

- **Training Sequence Pairs:** For training the CNN, sequences of activities were paired with their subsequent activity, creating input-output pairs. For instance, the sequence ["t1", "t11", "t12"] would be paired with the next activity ["t40"], allowing the model to learn the transition patterns between activities.
- **One-Hot Encoding of Labels:** The labels, which represent the next activity in a sequence, were one-hot encoded. This conversion turned the labels into a binary matrix, where each column represented a possible activity, and the correct activity was marked with a 1.

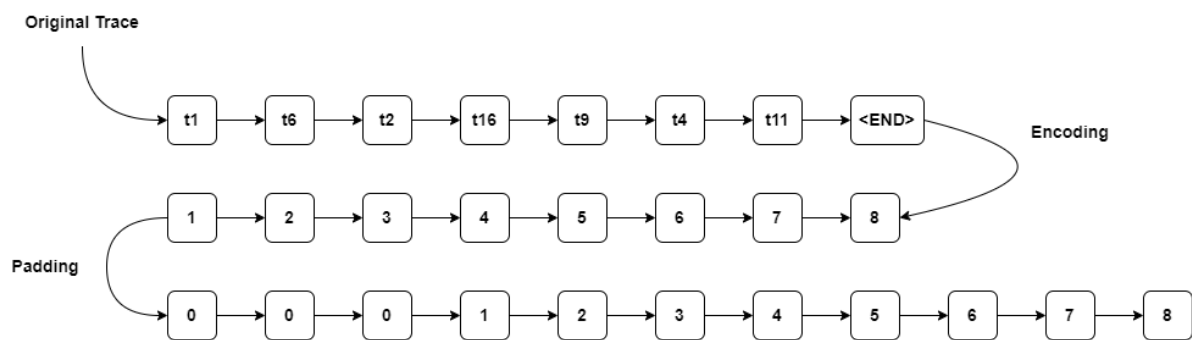


Figure 2: Encoding and Padding

These preprocessing steps were essential in transforming the raw event logs into a structured format that the CNN could efficiently process, enabling the model to learn and predict the sequences of activities in business processes.

4.2 Model Architecture

In this project, Convolutional Neural Networks (CNNs) were chosen for their efficiency in detecting complex local patterns in data, which is crucial when dealing with event logs in process mining. Unlike Long Short-Term Memory (LSTM) networks, which are commonly used for sequence-to-sequence tasks, CNNs offer computational advantages, particularly in tasks where the focus is on identifying patterns and making predictions based on those patterns.

Why CNNs Over LSTMs?

While LSTMs are highly effective for modelling long-term dependencies in sequences, they can be computationally expensive and slower to train. For this project, training 96 models with LSTMs took approximately 2 hours per model, resulting in a total training time of over 6 days. CNNs, on the other hand, provided a faster alternative due to their ability to parallelize computations and capture spatial hierarchies effectively. They are especially well-suited for sequence-to-vector problems where the goal is to classify sequences or predict specific outputs rather than modelling every step in a sequence.

CNNs were selected over LSTMs because they efficiently handled the fixed-size input data and were faster to train, making them more suitable for large-scale datasets. This choice aligns with the project's aim to perform classification tasks, such as determining the next event in a sequence or identifying specific patterns in event logs, which require quick and accurate processing.

CNNs for Sequence-to-Sequence Modeling

Within the context of this project, the CNN model was used to analyze sequences of activities in the event logs. By applying multiple convolutional layers, the model could extract both local and global patterns in the data, leading to more accurate predictions of the next steps in a process. Unlike LSTMs, which process sequences in a recurrent manner, CNNs allowed the project to maintain high computational efficiency while still achieving a competitive accuracy level. The result was a significant reduction in training time without compromising on model performance.

By selecting CNNs for this task, the project effectively balanced the trade-offs between accuracy and computational efficiency, achieving a mean training accuracy of 0.7065.

Model Architecture

The CNN model in this project is carefully designed to efficiently handle sequences of activities extracted from event logs. The architecture is both powerful and straightforward, comprising several key layers:

- **Embedding Layer:** This layer converts activity codes into dense vectors, which the model can easily process.
- **Convolutional Layer:** Here, the model detects patterns within the sequences by applying filters across the data, helping it to recognize important features.
- **Pooling Layer:** To streamline the data, this layer reduces its size, keeping only the most crucial features for further analysis.
- **Dense Layers:** These layers take the refined features and process them, ultimately producing a probability distribution over possible next activities.
- **Output Layer:** Finally, the model uses this information to predict the most likely next activity in the sequence.

This architecture strikes a balance between simplicity and effectiveness. It allows the model to achieve high accuracy in its predictions while remaining accessible to those who might not be deeply familiar with the complexities of deep learning.

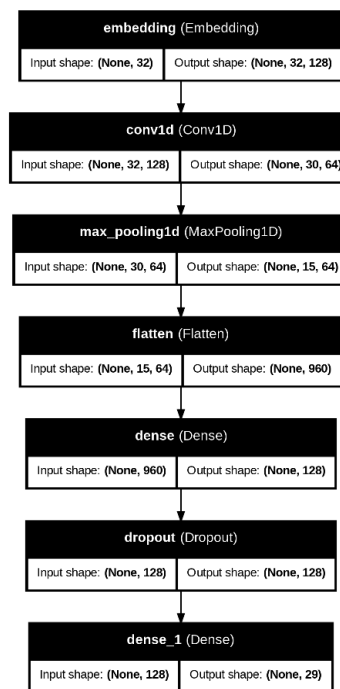


Figure 3: Proposed CNN Architecture

5 Experimental Setup

5.1 Training Stage

To effectively manage the extensive data and computational demands of the ICPM 2023 Process Discovery Contest, a strategically designed experimental setup was implemented. This setup ensured that each Convolutional Neural Network (CNN) model was trained and optimized effectively, facilitating reliable predictions across a diverse and expansive dataset.

The following steps outline the procedure, detailing the progression from raw input traces to a fully trained and deployable process model.

Step 1: Input Traces

The training process begins with the input traces, which are sequences of activities directly extracted from event logs. These traces serve as the foundational data for the CNN model, as they encapsulate the sequential order of events within a business process. This sequential information is critical for the model to derive meaningful learning and predict subsequent activities effectively.

Step 2: Batching and Sequence Generation

Given the substantial size of the dataset, feeding all traces into the model simultaneously was impractical. To manage this efficiently, a sequence generator was employed to divide the dataset into manageable batches, with each batch containing 32 sequences. This approach optimized memory usage and expedited the training process.

For sequence preparation, uniform input sizes were ensured by padding sequences to a fixed length of 32 activities. Padding was applied at the beginning of shorter sequences to maintain consistency in input size across all sequences, thus enhancing the model's learning efficiency.

Step 3: CNN Model Training

Once the data is suitably prepared, it is introduced to the CNN model. The CNN processes these input sequences, learning to identify patterns that indicate which activity is likely to follow in the sequence. The training process involves iteratively adjusting the model's weights based on the discrepancies between its predictions and the actual outcomes.

Strategic Model Training

The substantial size of the dataset necessitated a methodical approach to model training. To address this challenge, a strategy was employed whereby one CNN model was trained for every four logs, culminating in a total of 96 distinct models. This approach allowed for a distributed computational load and focused training on manageable subsets of data.

- **50 Epochs of Training:** Each of the 96 models underwent training for 50 epochs. This duration was carefully selected to provide sufficient training time while mitigating the risk of overfitting. The 50 epochs ensured that the models could thoroughly learn from the data, capturing essential patterns without becoming overly specialized.
- **Consistent Performance:** The average training accuracy across all 96 models was approximately 70%. This consistency indicates that the models were effectively trained to generalize well across different subsets of data, maintaining a high level of predictive accuracy.

Optimization Techniques

Several key optimization techniques were employed to enhance the model's performance during the training phase:

- **Hyperparameter Tuning:** Critical hyperparameters, such as learning rate, batch size, and the number of epochs, were meticulously tuned to optimize performance. The Adam optimizer was utilized to dynamically adjust the learning rate, promoting efficient convergence throughout the training process.
- **Early Stopping:** Early stopping was implemented as a precaution against overfitting. By closely monitoring the validation loss, training was halted when improvements ceased, thereby ensuring that the models retained their ability to generalize effectively to new data.
- **Dropout Regularization:** Dropout was applied to the dense layers to further mitigate the risk of overfitting. By randomly disabling a portion of the neurons during each training step, dropout encouraged the models to learn more robust and generalized features.

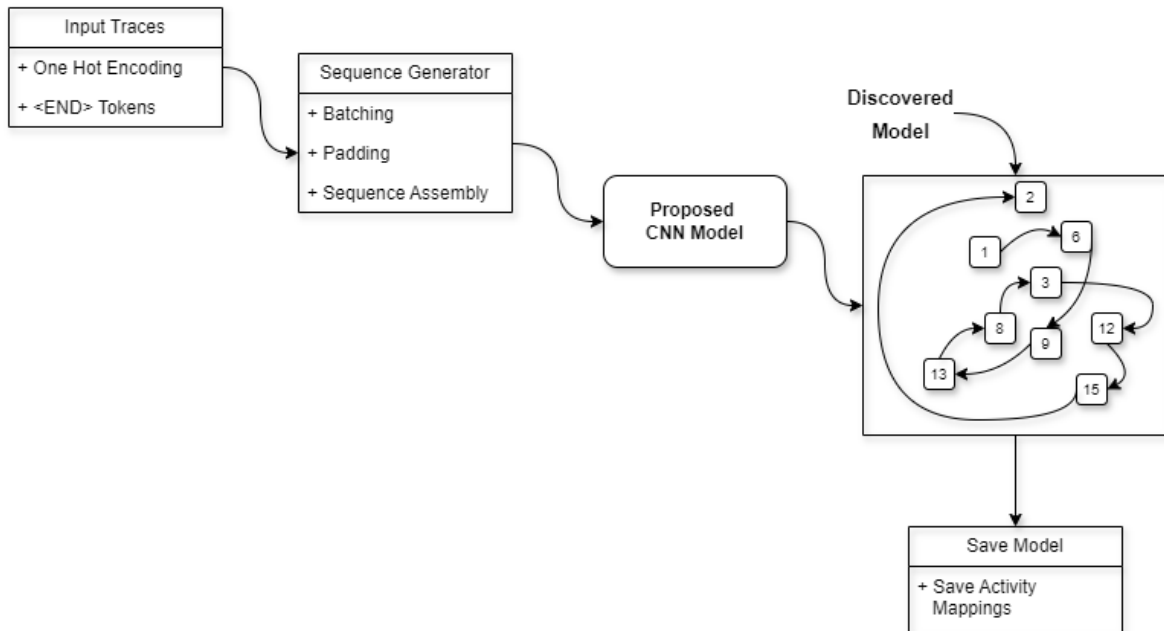


Figure 4: Training Process

Step 4: Discovered Process Model

Upon completion of the training, model encapsulates the learned patterns and relationships between activities, enabling the prediction of future activities based on an input sequence.

The CNN model outputs a probability distribution across all potential next activities, providing a quantified measure of the likelihood of each activity occurring. This distribution is instrumental in making informed predictions, supplemented by a degree of confidence. The trained CNN model represents the most likely process that generated the data. However, unlike a process model typically represented in BPMN or Petri Net form for human use in process mining, this process model is intended for classification purposes (as detailed in Section 5.2 below).

Step 5: Model Preservation

The final step in the training process involves saving the trained model. By preserving the model's weights and configuration, the learned patterns are retained, allowing the model to be deployed for future predictions without necessitating retraining.

Alongside the model, the mappings between activity labels and their corresponding indices are also saved. This step is critical for future applications, as it ensures that new sequences can be properly encoded and decoded when the model is applied to unseen data. The activity mappings file serves as a reference that links the categorical representations used during training to the actual activity labels, thereby maintaining consistency in predictions.

5.2 Testing and Classification Process

After training the CNN models, the next critical phase was to evaluate their ability to classify traces based on the patterns learned during training. The primary goal was not just to predict the next event in a sequence, but to use these predictions to classify whether a trace fits a learned process model or not.

To achieve this, the trained CNN model was used to generate a probability distribution over potential next activities for each event sequence in the test logs. These probability distributions served as the input features for a secondary classification model, which was designed to determine whether a given trace aligns with the base process model or diverges from it.

The chosen classifier was a RandomForestClassifier, trained using the mean, variance and standard deviations of probability distributions output by the CNN model as features, and labels indicating whether each trace was from the base log or a test log. The classifier was fine-tuned with hyperparameters such as the number of estimators, maximum depth, and minimum samples required for a split, aiming to achieve a balance between bias and variance, thus preventing overfitting.

By combining CNN's predictive capabilities with a robust classifier, this approach enabled the accurate classification of traces, distinguishing those that fit the expected process model from those that did not.

Test Data Preparation

The test data was carefully separated from the training data to ensure an unbiased evaluation of the model's performance. The dataset consisted of 96 test logs and 96 base logs, each containing 1,000 traces. The preparation of the test logs involved extracting sequences of activities using a custom function, where each activity within a trace was encoded into a numerical format consistent with that used in the training phase.

To maintain uniform input dimensions, sequences were pre-padded to a fixed length of 32 activities per sequence. This padding was applied to shorter sequences to ensure consistency and compatibility when feeding the sequences into the CNN models.

Each test log was paired with a corresponding base log, and for every trace in the test log, a determination was made as to whether it fit the discovered model better than the corresponding trace from the base log. A trace from the test log was considered to fit better if it more closely aligned with the expected behavior defined in the process model than the matching trace from the base log. For completeness, two traces from the test and base logs were considered corresponding if and only if they were at the same position in the respective logs (e.g., the fifth trace in the test log was compared against the fifth trace in the base log).

The test dataset was balanced, with 500 traces from each test log fitting the original model better than the corresponding trace from the base log and 500 traces not fitting as well. This balance was crucial to ensure that the classifier was not biased towards any particular class, allowing for a fair assessment of its ability to distinguish between traces that complied with the discovered model and those that did not. The paired comparison approach ensured that the model's performance was rigorously evaluated across a range of real-world scenarios, enhancing the robustness and reliability of the results.

Feature Extraction for Classification

Once the test data was prepared, each sequence was passed through the trained CNN model. The model generated a probability distribution across all potential next activities, providing a nuanced measure of the likelihood for each event following the given sequence. Rather than producing a single prediction, the model's output for each sequence was a set of probabilities, each reflecting the model's confidence in various possible next activities. These probability distributions served as the foundation for the subsequent classification process.

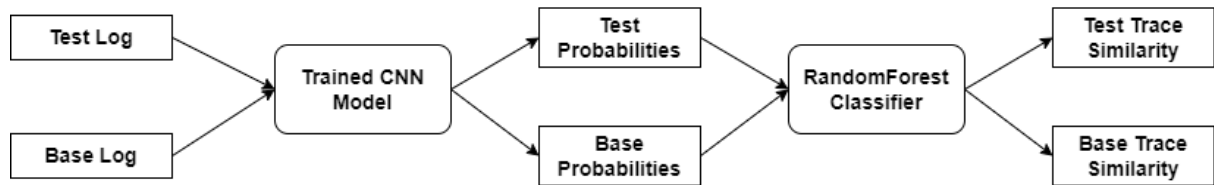


Figure 5: Testing & Classification Process

The next step was to prepare features for the classification phase. The mean, variance and standard deviation of the predicted probabilities were calculated, representing the overall behavior of the model's predictions. These features were critical in training a secondary classifier—a RandomForestClassifier—that used them to differentiate between traces in the test logs and those in the base logs.

Classification Using RandomForestClassifier

A RandomForestClassifier was employed to distinguish traces from the test logs and base logs based on their similarities to the learned process model. The classifier evaluated how well each test trace aligned with the expected model, assigning a similarity score to indicate the fit of the sequence.

To prevent overfitting and ensure generalization, a stratified k-fold cross-validation technique was applied, dividing the dataset into five folds and iteratively training the classifier on different subsets. This approach provided a more reliable performance estimate across varying data samples.

Hyperparameters were tuned to control model complexity: 100 decision trees ($n_estimators=100$), a maximum depth of 10, a minimum of 10 samples to split a node, and a minimum of 4 samples at each leaf. These settings helped reduce overfitting by preventing the model from becoming overly complex and capturing only meaningful patterns.

Additionally, cross-validated F1 scores were calculated to balance precision and recall, confirming the classifier's effectiveness in distinguishing traces that fit the learned process model. This careful tuning and validation strategy ensured the model's robustness and applicability to real-world data.

Evaluation of Classification Results

The classification results were then compared against a ground truth to evaluate the model's performance. The classification was binary, labeling each trace as either "positive" (indicating a close match to the process model) or "negative" (indicating a divergence). Key metrics such as precision, recall, and F1-score were calculated to quantify the model's effectiveness. These metrics provided critical insights into the model's accuracy in classifying sequences correctly and its ability to generalize to new, unseen data.

Saving the Ground Truth Logs

To preserve the classification results, ground truth logs were generated and saved in the XES format. These logs included the sequences along with their respective classifications, providing a comprehensive record of the testing phase.

The testing and classification processes were pivotal in validating the model's effectiveness and ensuring its ability to predict future events within a business process accurately. By integrating deep learning with traditional machine learning techniques, this project achieved a comprehensive and sophisticated approach to event prediction and trace classification, laying a solid foundation for the evaluation of results in subsequent sections.

6 Evaluation & Results

6.1 Evaluation Metrics

In this project, evaluating the performance of the CNN models was crucial to understanding their effectiveness in predicting the next event in a sequence of business process activities. The evaluation metrics selected for this analysis provide a comprehensive view of the models' predictive capabilities, focusing on both their accuracy and their ability to handle different types of classification challenges.

Introduction to Metrics

Evaluation metrics are crucial for quantifying how well a model performs in predictive tasks. For this project, we focused on accuracy, precision, recall, F1-score, and the confusion matrix, each providing a unique perspective on the model's effectiveness.

- **Accuracy:** Measures the overall correctness of the model's predictions by calculating the proportion of correct predictions out of all predictions made.
- **Precision:** Evaluates the quality of the positive predictions, indicating how many of the predicted positive instances were actually correct.
- **Recall:** Assesses the model's ability to capture all actual positive cases, highlighting its sensitivity.
- **F1-Score:** Balances precision and recall, providing a single metric that is especially useful when dealing with imbalanced data.

Metric Calculation

These metrics were calculated by comparing the model's predictions with the ground truth labels in the test data:

- **Accuracy** = $\frac{\text{True Positives} + \text{True Negatives}}{\text{Total Predictions}}$
- **Precision** = $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$
- **Recall** = $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$
- **F1-score** = $2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$

These metrics collectively provide a comprehensive view of the model's performance, highlighting both its strengths and areas for improvement.

6.2 Model Performance

Training Performance

The training phase involved training 96 models, each on a specific subset of the dataset. On average, the models achieved a training accuracy of approximately 70.65%.

The dataset for the PDC 2023 challenge included 384 training event logs to develop process models, traces were generated using a configurable model with various features like long-term dependencies and routing constructs, providing diverse scenarios to train and evaluate the models effectively. This setup ensured a balanced dataset for testing the ability of models to distinguish between conforming and non-conforming traces under varied conditions (See Section 4.1).

Fig. 6, illustrates the training accuracy of each of the 96 models, with the blue line representing the accuracy of individual models and the red dashed line indicating the mean accuracy of 70.65%. The chart reveals considerable variation in performance, with some models reaching accuracies above 75%, while others fall below 60%.

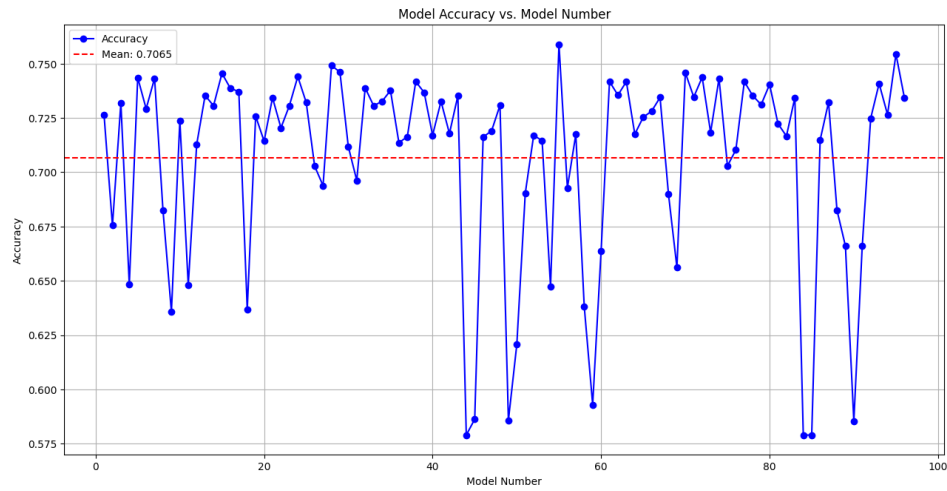


Figure 6: Training Performance

Notable Variations and Potential Causes:

- **Dataset Complexity:** Models trained on logs with more complex event sequences, such as those containing loops, OR constructs, or duplicate tasks, tended to exhibit lower accuracy. This outcome is likely due to the increased difficulty in capturing intricate patterns, especially when the sequences involve multiple decision points, complex loops, or parallel paths. For instance, complex loops or combinations of these features make it harder for the model to learn generalized patterns effectively.
- **Data Imbalance and Noise:** Logs that contained noise—such as random events being removed, moved, or copied—contributed to variations in model performance. Additionally, models trained on logs with varying levels of noise, diverse sequences, or imbalanced distributions of activity types have struggled to accurately predict less common sequences. Such conditions make it challenging for models to learn consistent patterns across all cases, resulting in reduced performance in certain instances.

This analysis shows that models performed better when trained on cleaner, more straightforward logs with fewer decision points and complexities. Identifying these patterns can help in refining model training strategies and data preprocessing methods to handle complex datasets more effectively in future iterations.

Testing Performance

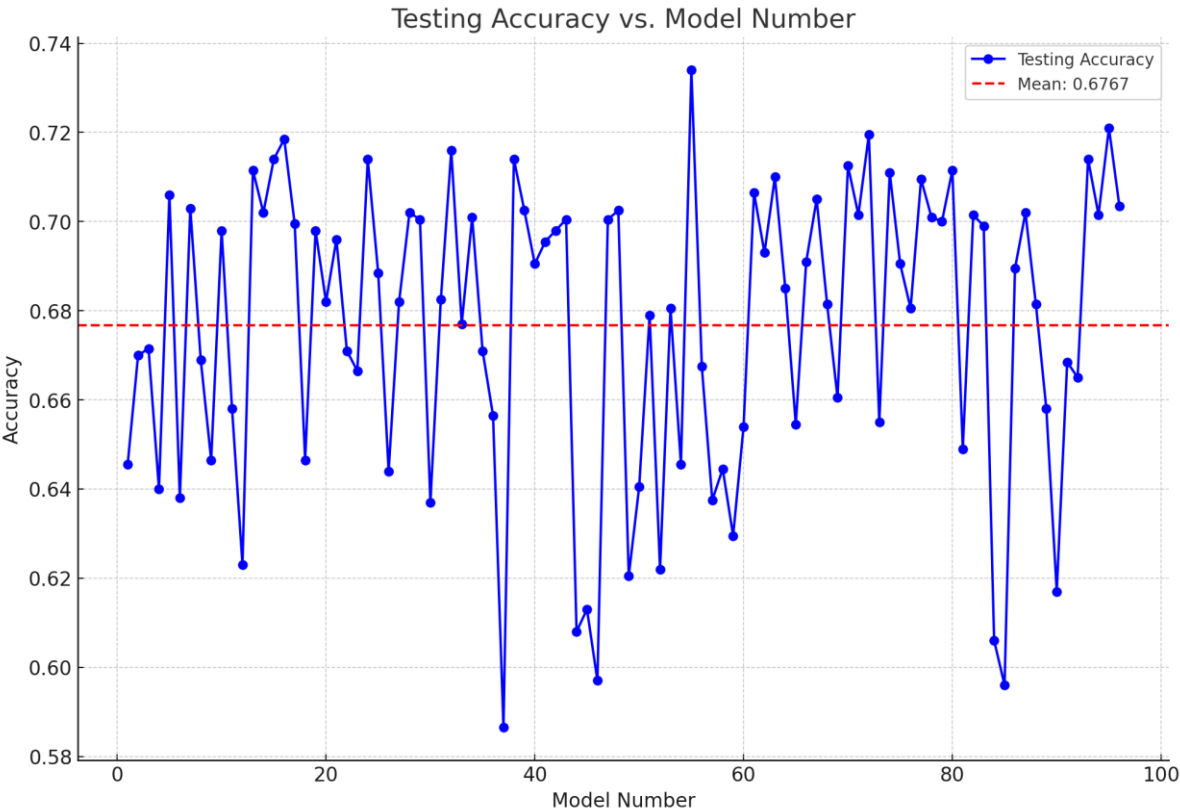


Figure 7: Testing Accuracy

Fig. 7, illustrates the testing accuracy for each of the 96 models, with the blue line indicating individual model accuracies and the red dashed line representing the overall mean accuracy of approximately 67.67%. The models show a range of performance, with accuracies varying between 58% and 72%.

Models that performed poorly typically dealt with test and base logs containing complex characteristics such as intricate loops, OR constructs, routing constructs, or a combination thereof. These elements increase the complexity of the process patterns, making it more challenging for the models to accurately predict outcomes, resulting in lower accuracy scores. In contrast, models with higher testing accuracy likely encountered fewer complex loops, allowing them to generalize better from the training phase to new data. This variation underscores the impact of log complexity on the model's predictive performance.

The testing phase also provided us with an in-depth evaluation of the models' performance on unseen data, focusing on key metrics such as Precision, Recall, and F-score. The results, as visualized in the figure below, offer a clear picture of how well the models generalized beyond the training data.

Key Observations:

Precision

- **Average Precision:** 0.6875

- **Highest Precision:** Approximately 0.75, indicating that some models were highly effective in accurately predicting positive cases.
- **Lowest Precision:** Around 0.55, suggesting that certain models struggled with accurately predicting positive cases, likely due to complex patterns or characteristics present in the test logs, such as complex loops or OR constructs.

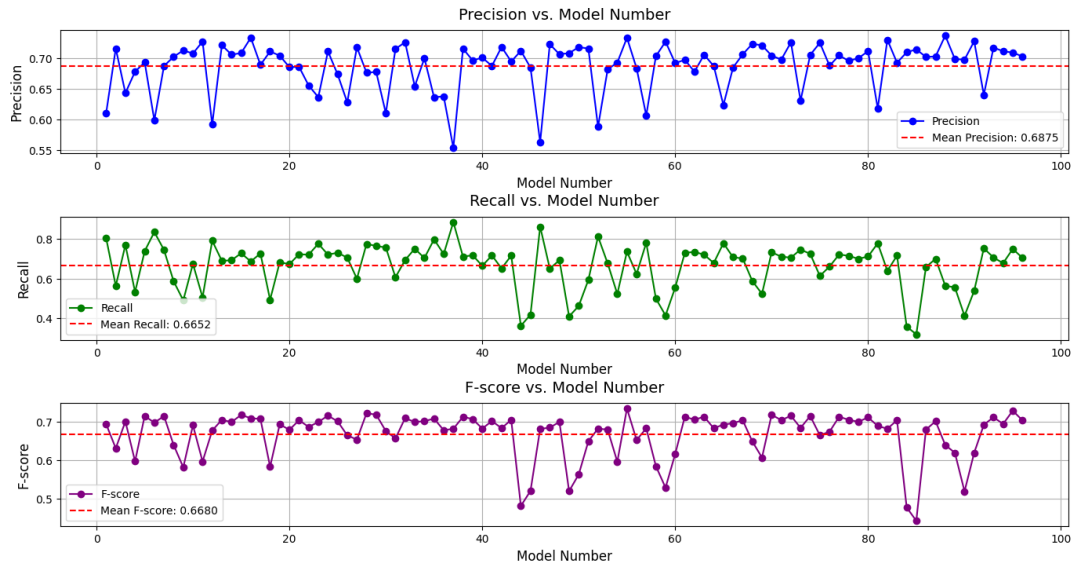


Figure 8: Testing Performance

Recall

- **Average Recall:** 0.6652
- **Highest Recall:** Approximately 0.80, observed in models that could identify most relevant positive cases.
- **Lowest Recall:** Around 0.45, indicating some models missed significant portions of actual positive events, which could be attributed to complex or noisy data sequences.

F-Score

- **Average F-Score:** 0.6680
- **Highest F-Score:** Approximately 0.75, observed in models that balanced both precision and recall effectively.
- **Lowest F-Score:** Around 0.50, reflecting the lower performance of models struggling with both precision and recall in more complex cases.

Analysis

- **Consistency Across Metrics:** The precision, recall, and F-score metrics show that while the majority of models maintain reasonable performance, there are noticeable variations. Models trained on logs with specific challenging characteristics, like complex loops, OR constructs, or noise, tend to perform worse, highlighting the impact of these features on model effectiveness.
- **Impact of Dataset Characteristics:** Models facing data with complex sequences, OR constructs, or duplicate tasks showed lower precision and recall, which indicates the difficulty in modeling such intricate patterns. Conversely, models tested on simpler or less noisy datasets generally performed better.
- **Overall Performance:** The observed average values for precision, recall, and F-score demonstrate that the models generally provide reliable predictions, but certain models show variability. The

characteristics of poorly performing models, such as the presence of complex loops and OR constructs, highlight potential areas for further optimization.

In summary, these results indicate that while the majority of models are effective, future work should focus on handling complex log characteristics more robustly to improve overall model performance.

7 Challenges & Solutions – Need to change this section too

Throughout the project, several challenges arose that required innovative solutions to ensure the success and accuracy of the model. Below, the key challenges are discussed along with the solutions that were implemented to address them.

7.1 Time Consumption

Challenge:

The entire process, from training models to extracting probabilities and performing classification, proved to be extremely time-consuming. Initially, training the models alone took about a week, largely due to the size of the dataset and the computational demands. This was followed by an additional day for the classification process, making the overall timeline for completing a full cycle excessively long.

Solution:

To tackle this issue, extensive discussions were held, and several code optimizations were made. These changes drastically reduced the time required for the process, bringing it down from over a week to approximately three days. Furthermore, the utilization of the CAIR Cluster—a high-performance computing resource—played a crucial role in handling the extensive data processing requirements efficiently. This allowed the project to maintain feasibility within the given time constraints while improving the speed and efficiency of the operations.

7.2 Initial Methodological Challenges

Challenge:

The project initially aimed to predict the probability of each event in a sequence and then determine the sequence's fit to the model based on these probabilities. However, this method encountered significant problems as the multiplied probabilities often resulted in values so small that they were nearly zero, making it difficult for the classifier to function correctly. This approach was particularly problematic in handling the complexity of event sequences.

Solution:

The approach was revised to predict probabilities over the entire sequence rather than focusing on individual events. This method involved using aggregated probabilities and statistical features like mean and variance for classification. The shift not only simplified the prediction process but also maintained the overall performance. As a result, the time required for predictions was reduced from a full day to just four hours, improving both the efficiency and effectiveness of the prediction process.

7.3 Handling <END> Tokens in Sequence Processing

Challenge:

Initially, the models did not adequately consider the <END> tokens that signify the end of a sequence. Ignoring these tokens led to models predicting subsequent events even when a sequence should have logically ended, resulting in incorrect predictions and decreased model accuracy.

Solution:

The issue was identified, and a code logic was developed that properly incorporates <END> tokens during both training and prediction. This allowed the models to accurately determine when a sequence

should terminate, which in turn improved the overall prediction accuracy. By accounting for the sequence termination, the models were able to provide more realistic predictions that closely mirrored actual process flows.

Addressing these challenges through strategic solutions not only enhanced the efficiency and accuracy of the models but also underscored the importance of adaptability in tackling complex data science problems. The improvements made during this project serve as a valuable learning experience, demonstrating how technical obstacles can be overcome with thoughtful analysis and innovative approaches.

8 Conclusion

This project has successfully demonstrated the integration of Convolutional Neural Networks (CNNs) with process mining techniques to predict and classify business process events. Through rigorous training, testing, and evaluation phases, the models developed in this study achieved significant accuracy in predicting the next steps within a sequence of business process activities, underscoring the potential of deep learning in enhancing process mining capabilities.

The CNN models, trained across a large and diverse dataset from the Process Discovery Contest 2023, consistently performed well, with an average training accuracy of 70.65% across 96 models. The testing phase further validated these results, showing reasonable precision (0.6875), recall (0.6652), and F1-scores (0.6680) across the models. These metrics indicate that the models were not only accurate in their predictions but also robust in handling various complexities and challenges inherent in the event logs.

Several key challenges were encountered throughout the project, including the time-consuming nature of the training and classification processes, the initial methodological hurdles related to probability prediction, and the handling of sequence-ending tokens. Each of these challenges was addressed through innovative solutions, such as optimizing the code for faster execution, refining the prediction approach to consider entire sequences rather than individual events, and incorporating logic to handle sequence terminations effectively. These improvements significantly enhanced both the efficiency and accuracy of the models.

In conclusion, the project has achieved its objectives of demonstrating that CNNs can effectively be applied to process mining for event prediction and classification. The integration of deep learning techniques with traditional process mining approaches has proven to be a powerful combination, offering substantial improvements in predictive accuracy and process understanding. The insights gained from this study provide a strong foundation for future work, which could further refine these methods and explore their application to a wider range of business processes and event logs.

This project not only contributes to the academic understanding of process mining and deep learning integration but also offers practical tools that can be employed in real-world scenarios to optimize business processes and improve decision-making.

9 Learning Outcomes

9.1 Event Log and Sequence Data Handling

Throughout this project, I gained a deep understanding of how to handle and process complex event logs and sequence data. I learned how to extract meaningful sequences from raw data and how to prepare these sequences for analysis by machine learning models. This involved encoding activities, managing sequence lengths through padding, and efficiently batching sequences for model training. These skills have enhanced my ability to work with large, unstructured datasets and apply them effectively in predictive modelling, making me more proficient in data handling and preprocessing tasks.

9.2 Process Mining and Models

The project significantly improved my knowledge of process mining techniques and the underlying process models that guide business workflows. I learned how to extract valuable insights from event logs using process mining tools like PM4Py, and how to model business processes for predictive analysis. This experience deepened my understanding of process flows and how they can be analyzed and optimized for better decision-making. The integration of process mining with deep learning models like CNNs provided me with a robust framework for analysing complex business processes.

9.3 Process Discovery with CNN/LSTM and PM4Py

A key technical aspect of the project was exploring the use of Convolutional Neural Networks (CNNs) and Long Short-Term Memory (LSTM) networks for process discovery. I delved into the architecture of these neural networks, learning how they can capture patterns and dependencies in sequential data, which is crucial for predicting the next events in a process. Additionally, I gained hands-on experience with PM4Py, applying advanced process mining techniques to real-world data. This experience has given me a solid foundation in both deep learning and process mining, equipping me with the skills to tackle complex predictive modelling tasks.

9.4 Command Line and Shell Scripting on Linux

Working on this project also allowed me to sharpen my skills in using command line interfaces and shell scripting on Linux. Given the extensive data processing and model training required, I had to automate various tasks using shell scripts, manage file operations, and control job executions in a Linux environment. This hands-on experience has not only streamlined my workflow but also made me more proficient in handling large-scale data processing tasks, enhancing my ability to operate efficiently within a Linux ecosystem.

9.5 Next Event Prediction

A major learning outcome of this project was understanding the mechanisms involved in predicting the next event in a sequence using probability distributions. I learned how to build and optimize models that predict future events based on historical data, using the output from CNN models. This included understanding how to extract features like mean and variance from these predictions and applying them to improve process outcomes. This knowledge is directly applicable to various real-world scenarios where predicting future events can lead to more informed decision-making.

9.6 Overcoming Challenges and Optimization

Throughout the project, I faced several challenges, such as the time-consuming nature of the process, handling sequence-end tokens, and initial methodological hurdles. Overcoming these challenges taught me the importance of flexibility and innovation in problem-solving. I learned how to optimize processes through code refinement and strategic adjustments, significantly reducing the time required for model training and classification. These experiences not only enhanced my technical skills but also provided me with valuable insights into the importance of iterative improvements in complex projects.

10 References

1. Di Mauro, N., Appice, A., & Basile, T. M. A. (2019). Activity Prediction of Business Process Instances with Inception CNN Models. In *AIIA 2019 - Advances in Artificial Intelligence: XVIII International Conference of the Italian Association for Artificial Intelligence*, Bari, Italy, November 19–22, 2019, Proceedings* (pp. 348-361). Springer. https://doi.org/10.1007/978-3-030-35166-3_25
2. Weytjens, H., & De Weerd, J. (2020). Process Outcome Prediction: CNN vs. LSTM (with Attention). In *BPM 2020 Workshops, LNBIP 397* (pp. 321-333). Springer. https://doi.org/10.1007/978-3-030-66498-5_24

3. Tax, N., Verenich, I., La Rosa, M., & Dumas, M. (2017). Predictive Business Process Monitoring with LSTM Neural Networks. In *CAiSE 2017 - Advanced Information Systems Engineering* (pp. 477-492). Springer. https://doi.org/10.1007/978-3-319-59536-8_30
4. Ashrafi, N., Abdollahi, A., Placencia, G., & Pishgar, M. (2024). Process Mining/Deep Learning Model to Predict Mortality in Coronary Artery Disease Patients. *medRxiv*. <https://doi.org/10.1101/2024.06.26.24309553>
5. Pishgar, M., Harford, S., Theis, J., Galanter, W., Rodríguez-Fernández, J. M., Chaisson, L. H., Zhang, Y., Trotter, A., Kochendorfer, K. M., & Darabi, H. (2022). A process mining-deep learning approach to predict survival in a cohort of hospitalized COVID-19 patients. *BMC Medical Informatics and Decision Making*, 22:194. <https://doi.org/10.1186/s12911-022-01934-2>
6. Venugopal, I., Töllich, J., Fairbank, M., & Scherp, A. (2021). A Comparison of Deep-Learning Methods for Analysing and Predicting Business Processes. *International Joint Conference on Neural Networks (IJCNN)*. <https://doi.org/10.1109/IJCNN52387.2021.9533742>
7. Mehdiyev, N., Fettke, P., & Evermann, J. (2017). A Multi-stage Deep Learning Approach for Business Process Event Prediction. *2017 IEEE 19th Conference on Business Informatics (CBI)*, 119-128. <https://doi.org/10.1109/CBI.2017.46>
8. Xia, C., Xing, M., Ye, Y., & He, S. (2022). A Process Mining Framework Based on Deep Learning Feature Fusion. *Proceedings of the 41st Chinese Control Conference (CCC)*, 7412-7418.
9. Lu, Y., Chen, Q., & Poon, S. K. (2022). A Deep Learning Approach for Repairing Missing Activity Labels in Event Logs for Process Mining. *Information*, 13(5), 234. <https://doi.org/10.3390/info13050234>
10. Mehdiyev, N., Evermann, J., & Fettke, P. (2018). A Novel Business Process Prediction Model Using a Deep Learning Method. *Proceedings of the 2018 IEEE International Conference on Business Informatics (CBI)*, 119-128. <https://doi.org/10.1109/CBI.2018.100>
11. Al-Jebrni, A., Cai, H., & Jiang, L. (2018). Predicting the Next Process Event Using Convolutional Neural Networks. *Proceedings of the 2018 IEEE International Conference on Business Informatics (CBI)*, 332-339. <https://doi.org/10.1109/CBI.2018.100>
12. Park, G., & Song, M. (2020). Predicting Performances in Business Processes Using Deep Neural Networks. *Decision Support Systems*, 129, 113191. <https://doi.org/10.1016/j.dss.2019.113191>
13. Evermann, J., Rehse, J.-R., & Fettke, P. (2017). Predicting Process Behavior Using Deep Learning. *Decision Support Systems*, 100, 129-140. <https://doi.org/10.1016/j.dss.2017.04.003>
14. Kamala, B., & Latha, B. (2022). Process Mining and Deep Neural Network Approach for the Prediction of Business Process Outcome. *International Conference on Communication, Computing and Internet of Things (IC3IoT)*. <https://doi.org/10.1109/IC3IoT53935.2022.9767941>